Fpt University

TRƯỜNG ĐẠI HỌC FPT

Activity & Lifecycle

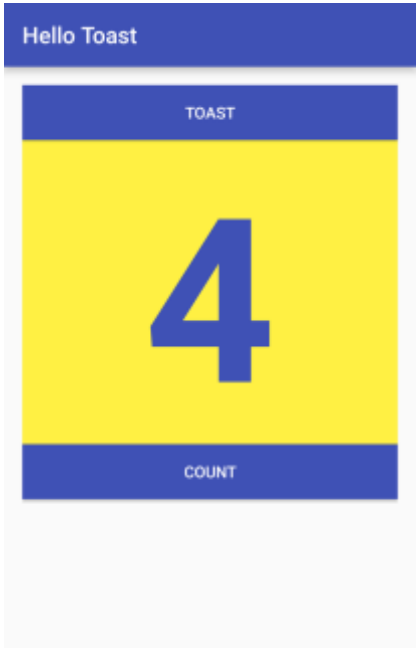
Contents

- Activities
- Defining an Activity
- Activity lifecycle
- Activity lifecycle callbacks

Activity

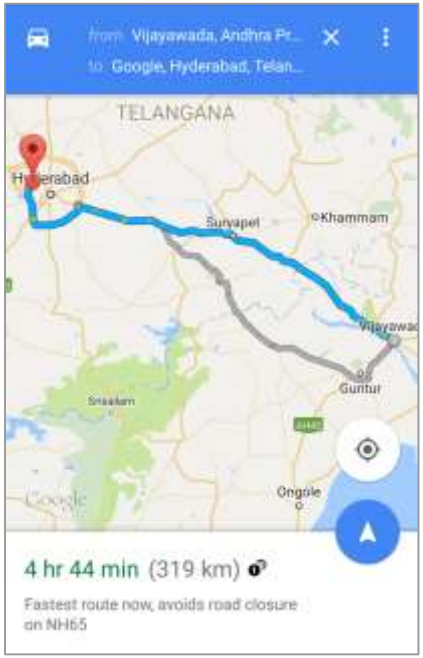
- An Activity is an application component
- Represents one window, one hierarchy of views
- Java class, typically one Activity in one file

Activity


Hello Toast
TOAST
4
COUNT


My Food List
Cheese
Pepperoni
Black Olives
Pineapple
Strawberries
Artichokes
Red peppers
Mushrooms


3.1415×2.718
8.538597
7 8 9 DEL
4 5 6 ÷
1 2 3 ×
. 0 = −
+


from Vijayawada, Andhra Pr...
to Google, Hyderabad, Telan...
TELANGANA
Suryapet
Khammam
Guntur
Ongole
4 hr 44 min (319 km)
Fastest route now, avoids road closure on NH65

Activity

- An Activity typically has a UI layout
- Layout is usually defined in one or more XML files
- Activity "inflates" layout as part of being created

Activity

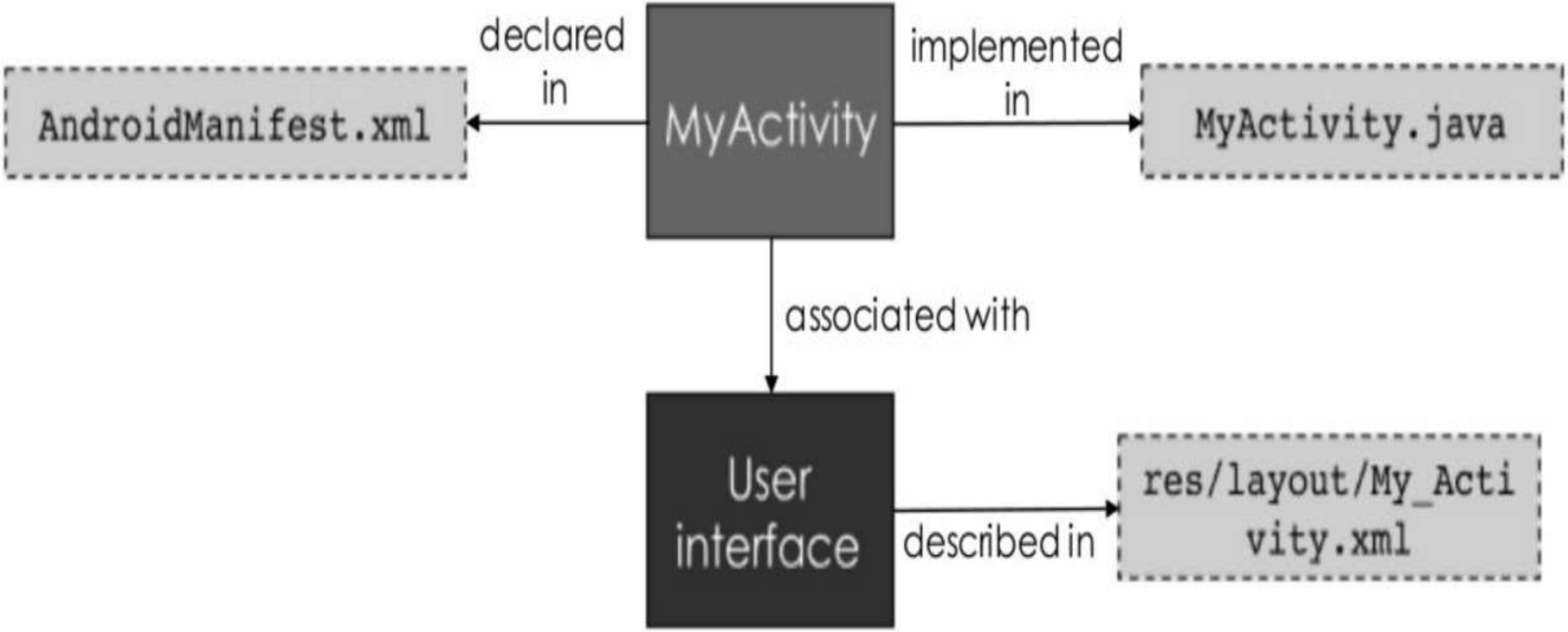
- **Activity is a necessary component in an Android application.**

- **MVC**

- **MVP**

- **MVVM**

Activity


declared in
AndroidManifest.xml
MyActivity
implemented in
MyActivity.java
associated with
User interface
described in
res/layout/My_Activity.xml

Implement new activities

- **Define layout in XML**
- **Define Activity Java class**
  - extends AppCompatActivity
- **Connect Activity with Layout**
  - Set content view in onCreate()
- **Declare Activity in the Android manifest**

Activity Implementation

- **Define Activity Java class**

```
public class ExampleActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState)
    {
        super.onCreate(savedInstanceState);
    }
}
```

Activity Implementation

- **Connect Activity with layout**

```
public class ExampleActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}
```

Activity Implementation

- **Declaring the activity in the manifest**

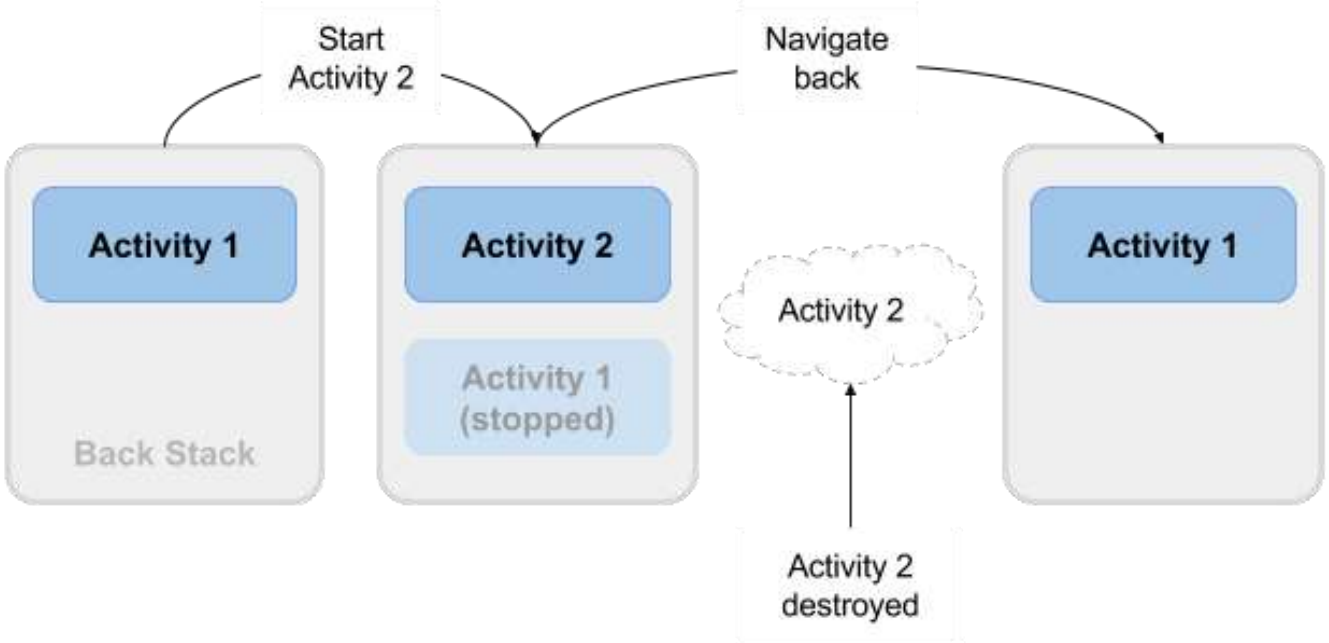
```
<manifest ... >
  <application ... >
      <activity android:name=".ExampleActivity" />
      ...
  </application ... >
  ...
</manifest >
```

- **Declaring the activity as a launch screen**

```
<activity android:name=".ExampleActivity" android:icon="@drawable/app_icon">
    <intent-filter>
        <action android:name="android.intent.action.MAIN" />
        <category android:name="android.intent.category.LAUNCHER" />
    </intent-filter>
</activity>
```

Activity Lifecycle

- The set of states an Activity can be in during its lifetime, from when it is created until it is destroyed

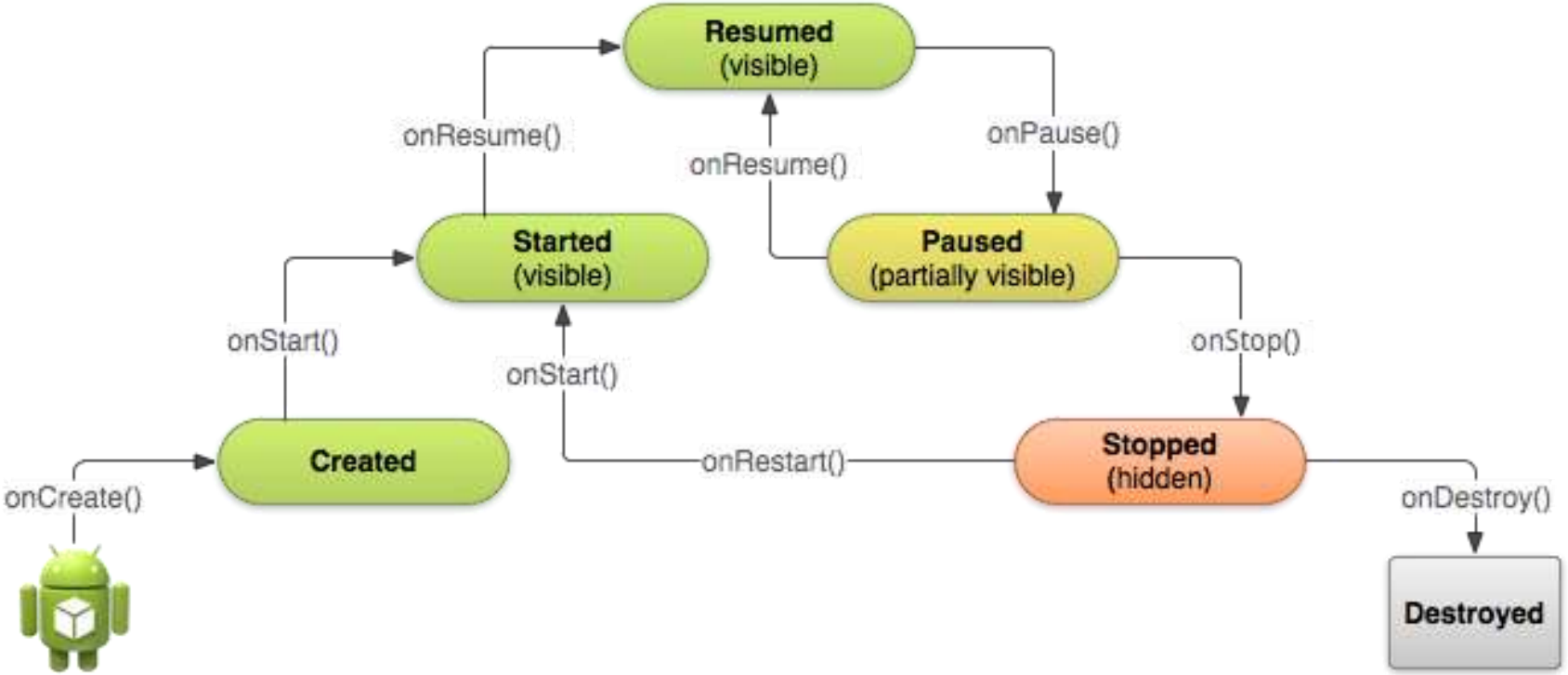


Activity states and app visibility

- **Created (not visible yet)**
- **Started (visible)**
- **Resume (visible)**
- **Paused(partially invisible)**
- **Stopped (hidden)**
- **Destroyed (gone from memory)**

State changes are triggered by user action, configuration changes such as device rotation, or system action

Activity states and callbacks graph



Activity Lifecycle callbacks

Method	Description
onCreate	called when activity is first created.
onStart	called when activity is becoming visible to the user.
onResume	called when activity will start interacting with the user.
onPause	called when activity is not visible to the user.
onStop	called when activity is no longer visible to the user.
onRestart	called after your activity is stopped, prior to start.
onDestroy	called before the activity is destroyed.

onCreate() –> Created

- Called when the Activity is first created, for example when user taps launcher icon
- Does all static setup: create views, bind data to lists, ...
- Only called once during an activity's lifetime
- Takes a Bundle with Activity's previously frozen state, if there was one
- Created state is always followed by onStart()

onCreate(Bundle savedInstanceState)

```
@Override
public void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    // The activity is being created.
}
```

onStart() –> Started

- Called when the Activity is becoming visible to user
- Can be called more than once during lifecycle
- Followed by onResume() if the activity comes to the foreground, or onStop() if it becomes hidden

onStart()

```
@Override
protected void onStart() {
    super.onStart();
    // The activity is about to become visible.
}
```

onRestart() –> Started

- Called after Activity has been stopped, immediately before it is started again
- Transient state
- Always followed by onStart()

onRestart()

```
@Override
protected void onRestart() {
    super.onRestart();
    // The activity is between stopped and started.
}
```

onResume() –> Resumed/Running

- Called when Activity will start interacting with user
- Activity has moved to top of the Activity stack
- Starts accepting user input
- Running state
- Always followed by onPause()

onResume()

```
@Override
protected void onResume() {
    super.onResume();
    // The activity has become visible
    // it is now "resumed"
}
```

onPause() –> Paused

- Called when system is about to resume a previous Activity
- The Activity is partly visible but user is leaving the Activity
- Typically used to commit unsaved changes to persistent data, stop animations and anything that consumes resources
- Implementations must be fast because the next Activity is not resumed until this method returns
- Followed by either onResume() if the Activity returns back to the front, or onStop() if it becomes invisible to the user

onPause()

```
@Override
protected void onPause() {
    super.onPause();
    // Another activity is taking focus
    // this activity is about to be "paused"
}
```

onStop() –> Stopped

- Called when the Activity is no longer visible to the user
- New Activity is being started, an existing one is brought in front of this one, or this one is being destroyed
- Operations that were too heavy-weight for onPause()
- Followed by either onRestart() if Activity is coming back to interact with user, or onDestroy() if Activity is going away

onStop()

```
@Override
protected void onStop() {
    super.onStop();
    // The activity is no longer visible
    // it is now "stopped"
}
```

onDestroy() –> Destroyed

- Final call before Activity is destroyed
- User navigates back to previous Activity, or configuration changes
- Activity is finishing or system is destroying it to save space
- Call isFinishing() method to check
- System may destroy Activity without calling this, so use onPause() or onStop() to save data or state

onDestroy()

```
@Override
protected void onDestroy() {
    super.onDestroy();
    // The activity is about to be destroyed.
}
```

Some methods related to activity lifecyle

- **finish()**
- OnBackPressedDispatcher

```
OnBackPressedDispatcher onBackPressedDispatcher = getOnBackPressedDispatcher();
onBackPressedDispatcher.addCallback(this, new OnBackPressedCallback(true) {
    @Override
    public void handleOnBackPressed() {
        // Handle back button press
    }
});
```

References

- https://google-developer-training.github.io/android-developer-fundamentals-course-concepts-v2/